

CyberAudit

JWT Authentication API

SimpleJWT · 60 min access · 60 day refresh · Token rotation · Blacklist

Project / Projet	CyberAudit Platform
Module	apps/accounts/ views.py + urls.py
Library	djangorestframework-simplejwt
Access token lifetime	60 minutes (timedelta(minutes=60))
Refresh token lifetime	60 days (timedelta(days=60))
Token rotation	ROTATE_REFRESH_TOKENS = True : new refresh on every refresh call
Blacklist on rotation	BLACKLIST_AFTER_ROTATION = True : old refresh token invalidated
Auth header	Authorization: Bearer <access_token>
User ID claim	user_id (USER_ID_CLAIM) : UUID in JWT payload
Throttle (login/register)	5 requests / minute (ScopedRateThrottle, scope='login')
Endpoints	8 endpoints across register / login / logout / refresh / me / change-password
Date	June 2026 / Juin 2026

1. JWT Concepts & SimpleJWT Strategy

1. Concepts JWT et strategie SimpleJWT

JSON Web Tokens (JWT) are self-contained signed tokens. The server signs the token with a secret key, any backend can verify the signature without a database lookup. CyberAudit uses a dual-token strategy: a short-lived access token (60 minutes) for API calls, and a long-lived refresh token (60 days) to obtain new access tokens without re-logging in.

Les JSON Web Tokens (JWT) sont des tokens signés auto-contenus. Le serveur signe le token avec une clé secrète, tout backend peut vérifier la signature sans requête DB. CyberAudit utilise une strategie a double token : un access token de courte duree (60 minutes) pour les appels API, et un refresh token longue durée (60 jours) pour obtenir de nouveaux access tokens sans se reconnecter.

Token lifecycle / Cycle de vie des tokens

Access token	60 min lifetime. Sent in Authorization header. Verified by JWTAuthentication on every request. NOT stored in DB stateless. 60 min de validité. Envoyé dans l'en-tête Authorization. Vérifié par JWTAuthentication a chaque requête. PAS stocké en DB sans état.
Refresh token	60 day lifetime. Sent in POST /token/refresh/ body. Stored in outstanding_token DB table. Blacklisted on use (rotation) and on logout. 60 jours de validité. Envoyé dans le corps POST /token/refresh/. Stocké dans la table DB outstanding_token. Blackliste à l'utilisation (rotation) et à la déconnexion.
Token rotation	ROTATE_REFRESH_TOKENS=True, every /token/refresh/ call issues a new refresh token and blacklists the old one. Prevents refresh token reuse. ROTATE_REFRESH_TOKENS=True, chaque appel /token/refresh/ émet un nouveau refresh token et blackliste l'ancien. Préviend la réutilisation du refresh token.
Blacklist	rest_framework_simplejwt.token_blacklist installed as Django app. Tables: outstanding_token + blacklisted_token. LogoutView calls token.blacklist() manually. rest_framework_simplejwt.token_blacklist installe comme app Django. Tables : outstanding_token + blacklisted_token. LogoutView appelle token.blacklist() manuellement.

JWT payload structure / Structure du payload JWT

```
# Decoded access token payload:
{
  'token_type': 'access',
  'exp': 1750000000,          # expiry timestamp (Unix)
  'iat': 1749996400,          # issued-at timestamp
  'jti': 'abc123...',         # JWT ID (unique per token)
  'user_id': '550e8400-e29b-41d4-a716-446655440000' # UUID (USER_ID_CLAIM)
}
# user_id is the UUID - JWTAuthentication uses it to load request.user
```

2. SimpleJWT Configuration : settings.py

2. Configuration SimpleJWT : settings.py

```
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), # access expires in 60 min
    'REFRESH_TOKEN_LIFETIME': timedelta(days=60), # refresh expires in 60 days
    'ROTATE_REFRESH_TOKENS': True, # new refresh issued on every
    /token/refresh/
    'BLACKLIST_AFTER_ROTATION': True, # old refresh immediately blacklisted
    'AUTH_HEADER_TYPES': ('Bearer',), # Authorization: Bearer <token>
    'USER_ID_FIELD': 'id', # User model field (UUID UUIDField)
    'USER_ID_CLAIM': 'user_id', # key name in JWT payload
}
```

REST_FRAMEWORK global auth + throttle

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ), # Every view uses JWT by default — overridden with AllowAny where needed
    'DEFAULT_PERMISSION_CLASSES': ('rest_framework.permissions.IsAuthenticated',),
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle', # 100/hour for anonymous
        'rest_framework.throttling.UserRateThrottle', # 300/hour for
authenticated
        'rest_framework.throttling.ScopedRateThrottle', # custom scopes
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/hour',
        'user': '300/hour',
        'login': '5/min', # used by RegisterView + LoginView
    },
}
```

INSTALLED_APPS : blacklist activation

```
INSTALLED_APPS = [
    ...,
    'rest_framework_simplejwt',
    'rest_framework_simplejwt.token_blacklist', # creates outstanding/blacklisted
tables
    'apps.accounts',
    ...
]
AUTH_USER_MODEL = 'accounts.User' # custom user model — UUID pk, email login
```

3. URL Routing : urls.py

3. Routage URL : urls.py

All auth routes are grouped under the prefix /api/auth/ (applied in config/urls.py). TokenRefreshView is the native SimpleJWT view, no custom code needed for token rotation.

Toutes les routes d'auth sont groupées sous le préfixe /api/auth/ (applique dans config/urls.py).

TokenRefreshView est la vue native SimpleJWT, aucun code personnalisé requis pour la rotation des tokens.

```
from rest_framework_simplejwt.views import TokenRefreshView
from .views import RegisterView, LoginView, LogoutView, MeView, ChangePasswordView

urlpatterns = [
    path('register/', RegisterView.as_view(), name='auth-register'),
    path('login/', LoginView.as_view(), name='auth-login'),
    path('logout/', LogoutView.as_view(), name='auth-logout'),
    path('token/refresh/', TokenRefreshView.as_view(), name='token-refresh'),
    path('me/', MeView.as_view(), name='auth-me'),
    path('change-password/', ChangePasswordView.as_view(), name='auth-change-
password'),
]
# Prefixed in config/urls.py: path('api/auth/', include('apps.accounts.urls'))
```

4. Endpoint Reference

4. Reference des Endpoints

URL	Method	Auth	Status codes	Description EN / FR
POST /api/auth/register/	POST	None	201, 400	Create account + return {access, refresh, user}. Créer un compte + retourner {access, refresh, user}.
POST /api/auth/login/	POST	None	200, 401, 403	Validate email/password + return tokens. 401=wrong creds, 403=inactive. Valider email/mdp + retourner tokens. 401=identifiants faux, 403=inactif.
POST /api/auth/logout/	POST	Bearer	204, 400	Blacklist the refresh token. Body: {refresh}. Blacklister le refresh token. Corps : {refresh}.
POST /api/auth/token/refresh/	POST	None	200, 401	Exchange refresh token for new access + new refresh (rotation). Body: {refresh}. Echanger le refresh contre un nouvel access + nouveau refresh (rotation). Corps : {refresh}.
GET /api/auth/me/	GET	Bearer	200, 401	Return authenticated user profile (no password). Retourner le profil de l'utilisateur connecté (sans mot de passe).
PATCH /api/auth/me/	PATCH	Bearer	200, 400, 401	Partial update: first_name / last_name / company_name only. Mise à jour partielle : first_name / last_name / company_name uniquement.
DELETE /api/auth/me/	DELETE	Bearer	204, 401	Soft-delete: sets is_active=False. Row preserved in DB. Suppression douce : passe is_active=False. Ligne préservée en DB.
POST /api/auth/change-password/	POST	Bearer	200, 400, 401	Verify old_password, hash and save new_password. Body:

				{old_password, new_password}. Vérifier old_password, hacher et enregistrer new_password. Corps : {old_password, new_password}.
--	--	--	--	---

POST /api/auth/register/ : request & response

```
# REQUEST body (JSON):
{
  'email':      'alice@example.com',
  'password':   'Security!',          // write_only, min 8 chars, validate_password
  'first_name': 'Alice',
  'last_name':  'Martin',
  'company_name': 'Test SARL'        // optional
}

# RESPONSE 201:
{
  'access':     'eyJhbGciOiJIUzI1NiIs... ', // 60 min
  'refresh':    'eyJhbGciOiJIUzI1NiIs... ', // 60 days
  'user': {
    'id':        '550e8400-e29b-41d4-a716-446655440000', // UUID
    'email':     'alice@example.com',
    'first_name': 'Alice',
    'last_name':  'Martin',
    'company_name': 'Test SARL',
    'role':       'client',
    'is_active':  true,
    'created_at': '2026-06-17T17:00:00Z'
  }
}
```

POST /api/auth/login/ : request & response

```
# REQUEST body:
{ 'email': 'alice@example.com', 'password': 'Security!' }

# RESPONSE 200: same shape as register — {access, refresh, user}

# RESPONSE 401 (wrong password OR unknown email — same message, anti-enumeration):
{ 'non_field_errors': ['Email ou mot de passe incorrect.'] }

# RESPONSE 403 (account deactivated):
{ 'non_field_errors': ['Ce compte est desactive.'] }
```

POST /api/auth/token/refresh/ : token rotation

```
# REQUEST body:
{ 'refresh': 'eyJhbGciOiJIUzI1NiIs...' }

# RESPONSE 200 (SimpleJWT native, ROTATE_REFRESH_TOKENS=True):
{
  'access': 'eyJ...NEW_ACCESS...', // new 60-min access token
  'refresh': 'eyJ...NEW_REFRESH...' // new 60-day refresh token
}
# Old refresh token is BLACKLISTED immediately (BLACKLIST_AFTER_ROTATION=True)
# Frontend must store the new refresh token, discarding the old one
```

POST /api/auth/logout/ : blacklist

```
# REQUEST headers: Authorization: Bearer <access_token>
# REQUEST body:
{ 'refresh': 'eyJhbGciOiJIUzI1NiIs...' }

# In LogoutView.post():
token = RefreshToken(refresh_token)
token.blacklist() # INSERT into blacklisted_token table

# RESPONSE 204: No Content
# RESPONSE 400: missing 'refresh' field OR token already blacklisted (TokenError)
```


5. View Implementations

5. Implementations des vues

`_token_response()` helper : shared by RegisterView + LoginView

```
def _token_response(user):
    refresh = RefreshToken.for_user(user) # creates refresh + embeds access
    return {
        'access': str(refresh.access_token), # short-lived (60 min)
        'refresh': str(refresh), # long-lived (60 days)
        'user': UserSerializer(user).data, # profile without password
    }
```

RegisterView + LoginView : AllowAny + throttle_scope='login'

```
class RegisterView(APIView):
    permission_classes = [AllowAny]
    throttle_scope = 'login' # ScopedRateThrottle: max 5 req/min

    def post(self, request):
        serializer = RegisterSerializer(data=request.data)
        serializer.is_valid(raise_exception=True) # 400 on validation error
        user = serializer.save() # calls User.objects.create_user() → hashes pwd
        return Response(_token_response(user), status=status.HTTP_201_CREATED)

class LoginView(APIView):
    permission_classes = [AllowAny]
    throttle_scope = 'login' # same 5/min limit

    def post(self, request):
        email = request.data.get('email', '').strip().lower()
        password = request.data.get('password', '')
        try:
            user = User.objects.get(email=email)
        except User.DoesNotExist:
            return Response({'non_field_errors': ['Email ou mot de passe incorrect.']},
                            status=status.HTTP_401_UNAUTHORIZED) # anti-enumeration
        if not user.check_password(password):
            return Response({'non_field_errors': ['Email ou mot de passe incorrect.']},
                            status=status.HTTP_401_UNAUTHORIZED) # same message
        if not user.is_active:
            return Response({'non_field_errors': ['Ce compte est desactive.']},
                            status=status.HTTP_403_FORBIDDEN)
        return Response(_token_response(user))
```

LogoutView : blacklist the refresh token

```
class LogoutView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        refresh_token = request.data.get('refresh')
        if not refresh_token:
            return Response({'detail': "Le champ 'refresh' est requis."},
                            status=status.HTTP_400_BAD_REQUEST)

        try:
            token = RefreshToken(refresh_token)
            token.blacklist() # adds to blacklisted_token DB table
        except TokenError:
            return Response({'detail': 'Token invalide ou deja revoke.'},
                            status=status.HTTP_400_BAD_REQUEST)

        return Response(status=status.HTTP_204_NO_CONTENT)
```

MeView : GET / PATCH / DELETE

```
class MeView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response(UserSerializer(request.user).data) # 200

    def patch(self, request):
        serializer = UserSerializer(request.user, data=request.data, partial=True)
        serializer.is_valid(raise_exception=True) # 400 on invalid data
        serializer.save() # only writable fields updated
        return Response(serializer.data) # 200

    def delete(self, request):
        # Soft delete - row stays in DB, login returns 403
        request.user.is_active = False
        request.user.save(update_fields=['is_active']) # only this column
        return Response(status=status.HTTP_204_NO_CONTENT)
```

ChangePasswordView

```
class ChangePasswordView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        serializer = ChangePasswordSerializer(
            data=request.data,
            context={'request': request} # needed by validate_old_password
        )
        serializer.is_valid(raise_exception=True) # 400 if old_password wrong
        request.user.set_password(serializer.validated_data['new_password'])
```

```
request.user.save(update_fields=['password'])
return Response({'detail': 'Mot de passe modifie avec succes.'}) # 200
```

6. Frontend Token Management

6. Gestion des tokens cote frontend

The React frontend (AuthContext + authService.js) stores the access token in memory (React state) and the refresh token in localStorage. On every API request, the Axios interceptor attaches the Bearer token. When a 401 is received, the interceptor automatically calls POST /token/refresh/ to get a new access token and retries the original request.

Le frontend React (AuthContext + authService.js) stocke l'access token en memoire (etat React) et le refresh token dans localStorage. Sur chaque requete API, l'intercepteur Axios attache le token Bearer. Quand un 401 est reçu, l'intercepteur appelle automatiquement POST /token/refresh/ pour obtenir un nouvel access token et relance la requete originale.

Token storage strategy / Strategie de stockage des tokens

Token	Stored in	Why	Risk
Access (60 min)	React state (memory)	XSS cannot read memory. Lost on page refresh (by design). XSS ne peut pas lire la memoire. Perdu au refresh (voulu).	Low, short-lived, in-memory only. Faible, courte duree, en memoire uniquement.
Refresh (60 days)	localStorage	Persists across tabs + page reloads. Restored on mount. Persiste entre onglets + rechargements. Restaure au montage.	XSS readable, mitigated by CSP + no eval. Lisible par XSS, atténue par CSP + pas d'eval.

Axios interceptor : automatic token refresh

```
// api.js - response interceptor (simplified)
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true;
      const refresh = localStorage.getItem('cyberaudit:refresh');
      const { data } = await api.post('/auth/token/refresh/', { refresh });
      // data.refresh = NEW refresh token (rotation) - must save it
      localStorage.setItem('cyberaudit:refresh', data.refresh);
      setAccessToken(data.access); // update in-memory access token
      error.config.headers['Authorization'] = `Bearer ${data.access}`;
      return api(error.config); // retry original request
    }
  }
);
```

```
    }  
    return Promise.reject(error);  
  }  
);
```

Login / Logout flow summary / Resume du flux Login / Logout

Login	POST /login/ → receive {access, refresh, user} → store access in state, refresh in localStorage, user in context. POST /login/ → recevoir {access, refresh, user} → stocker access en état, refresh en localStorage, user en contexte.
API call	Axios attaches Authorization: Bearer <access> to every request automatically. Axios attache Authorization: Bearer <access> a chaque requête automatiquement.
Token expired	Interceptor catches 401, calls POST /token/refresh/, replaces tokens, retries. Transparent to UI. L'intercepteur capture le 401, appelle POST /token/refresh/, remplace les tokens, relance. Transparent pour l'UI.
Logout	POST /logout/ with refresh token → blacklisted → clear state + localStorage → redirect to /login. POST /logout/ avec refresh token → blacklist → vider state + localStorage → rediriger vers /login.
Page refresh	Access token lost (memory). Interceptor auto-fetches new access using stored refresh on first 401. Access token perdu (mémoire). L'intercepteur recupere automatiquement un nouvel access via le refresh stocke au premier 401.

7. Authentication Tests

7. Tests authentication

The test suite for the auth endpoints comprises 10 tests across 4 classes (TestRegisterView, TestLoginView, TestMeView, TestLogoutView). All tests use the `api_client` fixture (unauthenticated) and `auth_client` fixture (pre-authenticated with JWT header).

La suite de tests pour les endpoints d'auth comprend 10 tests en 4 classes (TestRegisterView, TestLoginView, TestMeView, TestLogoutView). Tous les tests utilisent la fixture `api_client` (non authentifié) et `auth_client` (pre-authentifié avec en-tête JWT).

Test	HTTP	Assertion EN / FR
test_register_returns_201_and_tokens	201	Response contains 'access', 'refresh', and user.email == payload email. La reponse contient 'access', 'refresh', et user.email == email du payload.
test_register_duplicate_email_returns_400	400	Duplicate email rejected by RegisterSerializer unique constraint. Email duplique rejete par la contrainte unique de RegisterSerializer.
test_register_missing_email_returns_400	400	Missing email field triggers serializer validation error. Champ email manquant déclenche une erreur de validation du sérialiseur.
test_login_returns_tokens_and_user	200	Correct credentials return access token + user.role == 'client'. Identifiants corrects retournent access token + user.role == 'client'.
test_login_wrong_password_returns_401	401	Wrong password returns 401 (same as unknown email, anti-enumeration). Mauvais mot de passe retourne 401 (identique a email inconnu, anti-enumeration).
test_login_inactive_account_returns_403	403	is_active=False account returns 403, distinguishable from 401. Compte is_active=False retourne 403, distinguable du 401.
test_get_me_authenticated	200	auth_client (Bearer token) gets back their own email in response. auth_client (token Bearer) reçoit son propre email en réponse.
test_get_me_unauthenticated_returns_401	401	No Authorization header → JWTAuthentication returns 401. Pas d'en-tete Authorization → JWTAuthentication retourne 401.

test_patch_me_updates_first_name	200	PATCH with {first_name: 'Pauline'} returns updated value. PATCH avec {first_name: 'Pauline'} retourne la valeur mise a jour.
test_logout_blacklists_token	204	Valid refresh token is accepted and blacklisted. Returns 204. Refresh token valide est accepte et blackliste. Retourne 204.
test_logout_missing_refresh_returns_400	400	POST /logout/ with empty body returns 400 (refresh field required). POST /logout/ avec corps vide retourne 400 (champ refresh requis).

auth_client fixture : pre-authenticated client

```
# conftest.py
@pytest.fixture
def auth_client(api_client, client_user):
    from rest_framework_simplejwt.tokens import RefreshToken
    refresh = RefreshToken.for_user(client_user)
    api_client.credentials(HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}')
    return api_client
# Used by: test_get_me_authenticated, test_patch_me_updates_first_name,
#         test_logout_blacklists_token, test_change_password_*
```

8. Security Summary / *Récapitulatif sécurité*

Feature	Detail EN / FR
Short-lived access token	60 min expiry, exposure window is small. Stateless verification (no DB lookup per request). Expiration 60 min, fenêtre d'exposition faible. Vérification sans état (pas de requête DB par requête).
Long-lived refresh token	60 days, stays in DB (outstanding_token). Blacklisted on logout + on every rotation. 60 jours, reste en DB (outstanding_token). Blackliste à la déconnexion + à chaque rotation.
Token rotation	Every /token/refresh/ issues new refresh + blacklists old. Prevents stolen refresh token reuse. Chaque /token/refresh/ émet un nouveau refresh + blackliste l'ancien. Préviend la réutilisation d'un refresh volé.
Anti-enumeration	LoginView: same 401 for unknown email and wrong password. Attacker cannot discover registered emails. LoginView : même 401 pour email inconnu et mauvais mot de passe. L'attaquant ne peut pas découvrir les emails enregistrés.
Rate limiting (login)	throttle_scope='login' on RegisterView + LoginView: 5 requests/minute. Mitigates brute-force. throttle_scope='login' sur RegisterView + LoginView : 5 requêtes/minute. Atténue le brute-force.
Bearer header only	AUTH_HEADER_TYPES=('Bearer',) — tokens not accepted in URL params or cookies. AUTH_HEADER_TYPES=('Bearer',) — tokens non acceptés en paramètres URL ou cookies.
UUID user_id claim	USER_ID_CLAIM='user_id' stores UUID, prevents IDOR on token-based user lookup. USER_ID_CLAIM='user_id' stocke l'UUID, prévient les IDOR sur la recherche d'utilisateur par token.
Soft delete + 403	is_active=False → LoginView returns 403 (not 401). Disabled accounts cannot authenticate. is_active=False → LoginView retourne 403 (pas 401). Les comptes désactivés ne peuvent pas s'authentifier.
CORS	Only Vercel + Railway origins allowed. CORS_ALLOW_CREDENTIALS=True for cookie-free JWT auth. Seules les origines Vercel + Railway autorisées. CORS_ALLOW_CREDENTIALS=True pour l'auth JWT sans cookie.
CSP	Django-CSP middleware: default-src 'self', no inline scripts allowed (mitigates XSS token theft). Middleware Django-CSP : default-src 'self', pas de scripts inline autorisés (atténue le vol de token XSS).